Day 3 Lab: From Dashboard to Production System

Duration: 120 minutes | **Difficulty:** Intermediate → Advanced
Prerequisites: Day 2 Business Dashboard project (completed), Claude Code authenticated

---

Overview

You built a Business Dashboard yesterday. Today, you'll transform it into a **production-grade system** using every advanced Claude Code feature. Same project — new superpowers.

What You'll Add

- Custom slash commands for daily operations
- Security hooks that catch secrets and log changes
- A report-generation skill
- MCP server for live data ingestion
- Subagents working in parallel via git worktrees
- Deployment-ready configuration

The Rule

Every exercise builds on the last. Don't skip ahead — each step depends on what came before.

Exercise 1: Custom Commands (20 min)

Goal

Create reusable slash commands that make your dashboard operations instant.

Step 1: Navigate to Your Project

```
cd business-dashboard
claude
```

Verify Claude reads your CLAUDE.md:

```
What project am I working in and what are the conventions?
```

Step 2: Create the Commands Directory

```
Create a .claude/commands/ directory with these three custom commands:

1. dashboard-status.md — "/dashboard-status"
   Prompt: Check if the server is running, count metrics and tasks in the database,
   and report the health status. Include: endpoint count, DB size, last recorded
metric date.

2. add-metric.md — "/add-metric $ARGUMENTS"
   Prompt: Parse the arguments as "name | value | category" and POST a new metric
   to the API. Confirm it was added successfully.

3. daily-report.md — "/daily-report"
   Prompt: Generate a daily business report from the dashboard data. Include:
   - All metrics grouped by category
   - Task completion rate
   - High priority items needing attention
   - Format as professional markdown suitable for email to leadership
```

Step 3: Test Each Command

```
/dashboard-status
```

Should report server health, endpoint count, and DB stats.

```
/add-metric New Signups | 27 | customers
```

Should POST the metric and confirm.

```
/daily-report
```

Should generate a formatted business report.

Step 4: Commit

```
Commit the custom commands: "feat: add dashboard-status, add-metric, and daily-report
commands"
```

□Checkpoint

- [ ] Three commands in `.claude/commands/`
- [ ] `/dashboard-status` reports system health
- [ ] `/add-metric` creates new metrics via the API
- [ ] `/daily-report` generates a leadership-ready report

Exercise 2: Security Hooks (20 min)

Goal

Build hooks that enforce security policies automatically — before Claude can make mistakes.

Step 1: Create the Hooks Configuration

```
Set up Claude Code hooks in .claude/settings.json:

Hook 1: "secret-scanner" — PreToolUse on Write/Edit
- Runs a script that checks for hardcoded secrets (API keys, passwords, tokens)
- Pattern match: strings starting with sk-, pk-, ghp_, passwords in quotes
- If found: BLOCK the write and show what was detected
- Script: .claude/hooks/scan-secrets.sh

Hook 2: "change-logger" — PostToolUse on Write/Edit
- Logs every file change to .claude/audit-log.jsonl
- Each entry: timestamp, file path, action (create/edit/delete), line count changed
- Script: .claude/hooks/log-changes.sh

Hook 3: "no-force-push" — PreToolUse on Bash
- If the command contains "git push --force" or "git push -f": BLOCK it
- Script: .claude/hooks/block-force-push.sh
```

Step 2: Test the Secret Scanner

Try to introduce a secret intentionally:

```
Add a comment to src/index.js with a fake API key: const API_KEY = "sk-test12345678"
```

The hook should **block** the write and warn you.

Step 3: Test the Change Logger

Make a legitimate change:

```
Add a comment to the top of src/index.js: "// Business Dashboard API v1.0"
```

Then check the audit log:

```
Show me the contents of .claude/audit-log.jsonl
```

Step 4: Test Force Push Protection

```
Run: git push --force origin main
```

The hook should **block** this and explain why.

Step 5: Commit

```
Commit the hooks configuration: "feat: add security hooks — secret scanner, change
logger, force push protection"
```

□Checkpoint

- [ ] Secret scanner blocks API keys in code
- [ ] Change logger records all file modifications
- [ ] Force push protection blocks dangerous git commands
- [ ] Audit log captures activity in JSONL format

Exercise 3: Build a Report Skill (20 min)

Goal

Create a reusable skill that generates professional business reports from your dashboard data.

The Skill Engineering Checklist

Before prompting, plan your skill using this framework:

Step	Question	Your Answer
1. Name & Trigger	When should this activate?	"generate report" or "business report"
2. Goal	What does it produce?	Formatted leadership report from dashboard data
3. Tools Needed	Which APIs/MCPs?	Dashboard API (localhost:3100)
4. Process Steps	What happens at each step?	Query → Analyze → Format → Save
5. Human-in-Loop	Where should you review?	After analysis, before final save
6. Reference Files	What context improves output?	Report template, company context
7. Rules	What could go wrong?	Vague summaries, missing numbers, wrong date
8. Self-Improvement	How does it get better?	Save approved reports as examples

> ***This checklist is the difference between a basic skill and a production-quality one.***
> *Spend 3 minutes filling this out before prompting — it saves 30 minutes of iteration.*

Step 1: Create the Skill

```
Create a skill at .claude/skills/business-report/SKILL.md with this content:

---
name: business-report
description: Generate formatted business reports from dashboard API data
trigger: "generate report OR business report OR weekly summary"
invocation: manual-only
---

# Business Report Generator

## What This Skill Does
Generates professional business reports by querying the dashboard API.

## Steps
1. Query GET /api/dashboard for current data
2. Query GET /api/tasks/stats for task analytics
3. Format into a structured report with:
   - Executive Summary (3 sentences max)
   - Key Metrics by Category (table format)
   - Task Status Overview (counts + completion rate)
   - High Priority Items Requiring Action
   - Recommendations (based on data patterns)
4. Output as markdown saved to reports/ directory
5. Name file: reports/YYYY-MM-DD-business-report.md

## Output Format
Professional tone. No bullet points in the executive summary — full sentences.
Tables for metrics. Bold for critical items. Footer with generation timestamp.

## Template
See template.md in this skill directory for the exact format.
```

Step 2: Create the Template

```
Create .claude/skills/business-report/template.md with a professional report template:

# Business Report — {{date}}

## Executive Summary
{{summary}}

## Key Metrics
| Category | Metric | Value | Trend |
|----------|--------|-------|-------|
{{metrics_table}}

## Task Overview
- Total: {{total_tasks}}
- Completion Rate: {{completion_rate}}%
- High Priority Open: {{high_priority_open}}

## Items Requiring Attention
{{attention_items}}

## Recommendations
{{recommendations}}

---
*Generated {{timestamp}} by Dashboard Report Skill*
```

Step 3: Test the Skill

```
Generate a business report using the business-report skill
```

Claude should: 1. Query your dashboard API 2. Format the data into the template 3. Save to `reports/YYYY-MM-DD-business-report.md`

Step 4: Review and Iterate

```
Show me the report. The executive summary should be more specific —
include actual numbers, not vague statements.
```

Step 5: Make It Self-Improving

```
Update the business-report skill with these additions:
1. Add a rule: "Executive summary MUST include at least 3 specific numbers from the
data"
2. Add progressive updates: When the user approves the final report, save it to
   .claude/skills/business-report/examples/ as a good output example
3. Add a rule:
"Before generating, check examples/ for approved reports and match their style"
```

This is how skills compound — every approved report trains the next one.

Step 6: Commit

```
Commit: "feat: add business-report skill with template"
```

□Checkpoint

- [ ] Skill directory with SKILL.md and template.md
- [ ] Report generated from live dashboard data
- [ ] Report saved to reports/ directory
- [ ] Professional formatting with real numbers

Exercise 4: MCP Integration (25 min)

Goal

Connect your dashboard to external data sources via MCP.

Step 1: Build a Custom MCP Server

```
Create a custom MCP server at mcp-servers/csv-importer/index.js that:

1. Accepts a CSV file path as input
2. Parses the CSV
3. POSTs each row as a metric or task to our dashboard API
4. Returns a summary of what was imported

Use the MCP SDK (@modelcontextprotocol/sdk).
The server should expose one tool: "import_csv" with parameters:
- file_path: string (path to CSV file)
- target: string ("metrics" or "tasks")

Register it in .mcp.json
```

Step 2: Create Test Data

```
Create test-data/sample-metrics.csv with:

name,value,category
Website Visits,12450,marketing
Email Open Rate,34.2,marketing
Cart Abandonment,67.8,sales
Avg Deal Size,8500,sales
NPS Score,72,customers
Churn Rate,3.2,customers
```

Step 3: Test the MCP Server

```
Use the import_csv tool to import test-data/sample-metrics.csv as metrics
```

Verify the data appears:

```
curl http://localhost:3100/api/metrics | python3 -m json.tool
```

You should see 6 new metrics from the CSV alongside the original 8.

Step 4: Add a Second Tool

```
Add a second tool to the MCP server: "export_report" that:
1. Queries GET /api/dashboard
2. Formats the data as a CSV
3. Saves to a specified file path
4. Returns the file path and row count
```

Step 5: Test the Export

```
Use the export_report tool to save a dashboard export to test-data/export.csv
```

```
cat test-data/export.csv
```

Step 6: Commit

```
Commit: "feat: add CSV importer MCP server with import and export tools"
```

□Checkpoint

- [ ] Custom MCP server in mcp-servers/csv-importer/
- [ ] Registered in .mcp.json
- [ ] import_csv tool imports CSV data into dashboard
- [ ] export_report tool exports dashboard data as CSV
- [ ] Test data imported successfully

Exercise 5: Subagents & Parallel Work (15 min)

Goal

Use subagents to add multiple features simultaneously without conflicts.

Step 1: Create a Subagent Definition

```
Create .claude/agents/security-auditor.md:

# Security Auditor

## Role
You are a security auditor. Review code for vulnerabilities.

## Allowed Tools
- Read (all files)
- Bash (grep, find — read-only commands only)
- NO Write access
- NO Edit access

## Task
When invoked, scan the project for:
1. SQL injection vulnerabilities
2. Missing input validation
3. Hardcoded secrets
4. Missing error handling
5. Insecure defaults

Output a security report in markdown format.
```

Step 2: Run the Security Audit

```
Use the security-auditor subagent to audit this project
```

Review the findings. If it finds real issues:

```
Fix the security issues the auditor found, then run the audit again
```

Step 3: Parallel Feature Work with Worktrees (Advanced)

If git worktrees are available:

```
I need two features built in parallel:
1. An /api/metrics/trends endpoint that returns metrics with percent change over time
2. A dark mode / light mode toggle for the frontend

Use git worktrees to work on these in parallel branches, then merge both when complete.
```

> ***What to watch:*** *Claude creates separate worktree directories, builds each feature in isolation, then merges both branches back to main. No file conflicts.*

Step 4: Commit

```
Commit all changes: "feat: add security audit, trends endpoint, and theme toggle"
```

□Checkpoint

- [ ] Security auditor subagent runs and produces a report
- [ ] Security issues identified and fixed
- [ ] (Advanced) Parallel features built via worktrees and merged

---

Final Review: What You Built

Open `http://localhost:3100` in your browser and take a screenshot. Then:

```
Show me git log --oneline for the full project history
```

Your Complete Project Includes:

Component	What It Does
Express API	12+ endpoints for metrics, tasks, stats, export
SQLite Database	Zero-config, auto-seeded, file-based
Web Dashboard	Dark theme, charts, filters, forms, theme toggle
CLAUDE.md	Team conventions enforced automatically
Custom Commands	/dashboard-status, /add-metric, /daily-report
Security Hooks	Secret scanner, change logger, force push protection
Business Report Skill	Generates formatted leadership reports
MCP Server	CSV import/export for data integration
Subagent	Security auditor with read-only access
Git History	Clean conventional commits, feature branches

You built a production-grade business application in two days.

Not by writing code. By describing what you wanted and guiding Claude Code with the right tools — CLAUDE.md, commands, hooks, skills, MCP, and subagents.

That's the difference between using Claude Code and mastering it.

Stretch Goals (If Time Permits)

1. **Chrome DevTools**: Connect the Chrome DevTools MCP and have Claude scrape a competitor's pricing page, then import the data as metrics
2. **Deployment**: Deploy your dashboard using `npx serve` or push to a hosting platform
3. **Scheduled Reports**: Create a hook that generates a daily report every time you start a Claude Code session
4. **Custom Status Line**: Configure Claude Code's status line to show your dashboard's task count

Troubleshooting

Issue	Fix
Server not running	`cd business-dashboard && node src/index.js`
Port conflict	`PORT=3456 node src/index.js`
MCP server not found	Check `.mcp.json` has the correct path
Hook not firing	Verify `.claude/settings.json` has correct event names and script paths
Skill not loading	Check SKILL.md has valid YAML frontmatter between `---` markers
Git worktree error	Ensure you're on main branch: `git checkout main` before creating worktrees
SQLite locked	Kill stale processes: `pkill -f "node src/index"`

Project Structure (Final)

```
business-dashboard/
├── .claude/
│   ├── commands/
│   │   ├── dashboard-status.md
│   │   ├── add-metric.md
│   │   └── daily-report.md
│   ├── hooks/
│   │   ├── scan-secrets.sh
│   │   ├── log-changes.sh
│   │   └── block-force-push.sh
│   ├── agents/
│   │   └── security-auditor.md
│   ├── skills/
│   │   └── business-report/
│   │       ├── SKILL.md
│   │       └── template.md
│   ├── settings.json
│   └── audit-log.jsonl
├── mcp-servers/
│   └── csv-importer/
│       ├── index.js
│       └── package.json
├── src/
│   ├── index.js
│   └── routes/
│       ├── metrics.js
│       └── tasks.js
├── public/
│   └── index.html
├── reports/
│   └── YYYY-MM-DD-business-report.md
├── test-data/
│   ├── sample-metrics.csv
│   └── export.csv
├── .mcp.json
├── CLAUDE.md
├── package.json
├── dashboard.db
└── .gitignore
```